

Homework 1: Linear Search and Binary Search

*Instructor: Harsh Parikh**Due: Saturday, September 26, 2026, 11:59 pm***Instructions**

Scope. This assignment builds two procedures and then asks four questions of them. The procedures are linear search and binary search. The questions are whether the procedure halts, whether it is correct, what it costs in time, and what it costs in space. Problem 1 implements both and measures them. Problems 2 to 4 answer the four questions by proof. Both procedures are given below. You supply the answers.

Grading. Four problems of five subproblems, 5 points each, 100 total. Points are for a genuine attempt with the reasoning shown, not for a correct answer. Written feedback is given on all work.

Submission. Upload three files to Canvas:

- `hw1-yourlastname.pdf`
- `hw1-yourlastname.tex`
- `hw1-yourlastname.py` (Problem 1)

A submission missing any of the three is incomplete. Write your solutions in `hw-solution-template.tex`, on the course website.

No handwritten work is accepted, figures included: use `tikz`, or generate the figure in Python and include the image file. (In-class quizzes are the exception.)

Late work. Not accepted. Contact me before the deadline if a documented emergency applies.

Collaboration. Discussion is permitted. Write your own solutions and list everyone you worked with.

AI use. Permitted, and must be declared at the top of your submission: the tool, what it did, what you verified, and what you did not. An undeclared use, or an inaccuracy traced to one, scores zero.

Instructor AI declaration. Claude Opus 5 (Anthropic) was used for auditing and polishing the homework. Report any error you find.

Getting started: Python and L^AT_EX

This section is for students who have not used Python or L^AT_EX before. Skip it if you already run Python and compile L^AT_EX. Nothing in it is graded, and none of it requires the command line.

Installing Python. Install **Anaconda**, from <https://www.anaconda.com/download>. Choose the graphical installer for your operating system and accept the defaults. Anaconda includes Python itself, Jupyter Notebook, and the two libraries this assignment uses, `matplotlib` and `numpy`. Installing it separately from any Python that came with your machine is fine and is the usual case.

Writing and running code. Open **Anaconda Navigator**, which the installer adds to your applications. Click **Launch** under Jupyter Notebook. A page opens in your browser showing your files. Use **New**, then **Python 3**, to create a notebook.

A notebook is a list of *cells*. Type code into a cell and press **Shift** and **Enter** together to run it. Anything the cell prints appears underneath it. Variables and functions defined in one cell are available in the cells you run afterwards, so you can define a procedure in one cell and call it in the next. If results stop making sense, use **Kernel**, then **Restart and Run All**, which runs every cell in order from a clean start.

Producing the .py file. Canvas expects `hw1-yourlastname.py`, not a notebook. In Jupyter, open the **File** menu and choose the entry that exports the notebook as an executable script or as Python (.py); the exact wording depends on your Jupyter version. Rename the downloaded file to `hw1-yourlastname.py`. Open it once to confirm it contains your code.

Making the figure. Problem 1(d) asks for one plot. In a cell:

```
import matplotlib.pyplot as plt
plt.loglog(ns, times_linear, marker='o', label='linear')
plt.xlabel('n'); plt.ylabel('seconds'); plt.legend()
plt.savefig('timing-plot.png', dpi=200, bbox_inches='tight')
```

`loglog` gives logarithmic scales on both axes. `savefig` writes an image file, which you then include in your L^AT_EX document with `\includegraphics`. For the straight-line fits in Problem 1(e), `numpy.polyfit` on the base-10 logarithms of your two columns returns the slope.

Writing L^AT_EX. Use **Overleaf**, at <https://www.overleaf.com>. It runs in a browser, so there is nothing to install. A free account is enough for this course.

Create an account. Choose **New Project**, then **Upload Project**, and upload the template from the course website:

`hw-solution-template.tex`

Press **Recompile** to produce the PDF, which appears beside your text. Write your answers where the template marks them, recompiling as you go.

When you are finished, download two files from Overleaf: the PDF, and the source via **Menu**, then **Source**. Rename them `hw1-yourlastname.pdf` and `hw1-yourlastname.tex`.

If something breaks. A L^AT_EX error is almost always on the line Overleaf names, or just above it, and is usually a missing brace or a missing `\end`. The template compiles as given, so if it stops compiling, the last thing you typed is the cause. Undo it, recompile, and put it back in smaller pieces.

For anything else, come to office hours. Setting up tools is not what this assignment is testing, and you should not lose time to it.

Reference: the searching problem and the two procedures

Every problem in this assignment refers to this page. Problem 1 implements these two procedures and measures them. Problems 2 to 4 ask, in order: does it halt, is it correct, and what does it cost in time and in space.

The searching problem

Let A be an array of n records, indexed $A[0], \dots, A[n-1]$, drawn from a totally ordered set, and let t be a target value. A procedure *solves the searching problem* if it returns an index i with $A[i] = t$ whenever such an i exists, and returns NOT-FOUND otherwise.

Cost is counted in **comparisons**: each examination of one array element against t counts as one. **Space** is counted as memory used *beyond the array itself*. This includes the memory held by recursive calls that have not yet returned.

Linear search makes no assumption about the order of A . **Binary search** assumes A is sorted: $A[0] \leq A[1] \leq \dots \leq A[n-1]$.

In both procedures, lo and hi are indices. They bound the portion of A still under consideration, and both ends are included. The portion is empty when $lo > hi$. A search of the whole array is the call `linear(A, 0, n-1, t)` or `binary(A, 0, n-1, t)`. The operator `//` is integer division, rounding down.

Both procedures return NOT_FOUND when t does not occur. It is a sentinel you define once, above the two procedures: `NOT_FOUND = None`. Do not use `-1` for it. In Python `-1` is a valid index, so a caller that forgot to check the result would silently read the last record instead of reporting a failed search.

```
def linear(A, lo, hi, t):
    if lo > hi:
        return NOT_FOUND
    if A[lo] == t:
        return lo
    return linear(A, lo+1, hi, t)
```

```
def binary(A, lo, hi, t):
    if lo > hi:
        return NOT_FOUND
    mid = (lo + hi) // 2
    if A[mid] == t:
        return mid
    if A[mid] > t:
        return binary(A, lo, mid-1, t)
    return binary(A, mid+1, hi, t)
```

Refer to these two procedures throughout. Where a problem asks you to modify one, state the modification in full rather than describing the edit.

Problem 1. Implementation and measurement**[25 points]**

Parts (a)–(e) require code. Submit it as `hw1-yourlastname.py`; put tables, figures, and written answers in the PDF. Work from the procedures on the reference page. Do not use `in`, `.index()`, or `bisect`. Each of these already implements a search.

- (a) Implement `linear(A, lo, hi, t)` and `binary(A, lo, hi, t)` exactly as they are written on the reference page, recursively.

Test both on: the target at the first position; at the last; at an interior position; absent; an array of length 0; an array of length 1; and an array in which the target occurs several times. Report the seven tests and their results in a table, and state for each what you expected before you ran it.

Solution.

- (b) Add a counter to each procedure that records the number of comparisons against t , without changing what either returns.

For $n = 15$ and $A = \langle 0, 1, 2, \dots, 14 \rangle$, report the comparison count of each procedure for an absent target, and for the target at every one of the 15 positions. Give the two worst-case counts you observe, and every position that produces each.

Then, for each of $n = 1, 2, 3, 4, 7, 8$, report the largest comparison count `binary` makes over all targets, present and absent, on $A = \langle 0, 1, \dots, n-1 \rangle$. Give the six counts in a table.

Solution.

- (c) Run your recursive `linear` on an absent target for $n = 100, 200, 400, 800, 1600$. One of these sizes fails. Report the n at which it first fails, the error, and the reason, in one sentence.

Then run your recursive `binary` at $n = 2^{20} = 1,048,576$ and report whether it fails.

Rewrite both procedures as loops, `linear_loop` and `binary_loop`, returning the same values. Confirm on the tests from part (a) that they agree with the recursive versions, and confirm that `linear_loop` succeeds at the n where `linear` failed.

Solution.

- (d) Using the loop versions, measure the wall-clock time of a single search for an absent target at $n = 10^3, 10^4, 10^5, 10^6, 10^7$.

Repeat each measurement enough times that the value is stable, and state the number of repetitions and how you chose it. A single binary search is too fast to be timed reliably on its own. State how you handled this.

Produce one figure of time against n for both procedures, with logarithmic scales on both axes, labelled axes, a legend, and a caption.

Solution.

- (e) A running time of Cn^p appears on logarithmic axes as a line of slope p . Fit a line to each of your two datasets and report the two slopes.

State what slope you expected for each procedure and whether the measurements agree. State separately what a running time proportional to $\log n$ should look like on these axes, and whether your second dataset looks like it.

Identify any bend, step, or plateau in either dataset and give a probable cause. Two effects that do not appear anywhere in this assignment's analysis are the memory hierarchy and the cost of interpreting Python, and both are visible in measurements of this kind.

Solution.

Problem 2. Completeness: does the procedure halt?**[25 points]**

A recursive procedure halts on every input if it has a base case, and if every chain of recursive calls reaches one. The standard way to establish the second condition is a *measure*: a quantity, computed from the arguments, that is a non-negative integer, and that strictly decreases at every recursive call. A non-negative integer cannot decrease indefinitely, so the chain of calls is finite. It ends at a base case.

- (a) Give a measure for `linear` and a measure for `binary`. For each, state the quantity, verify that it is a non-negative integer whenever the procedure is called with $0 \leq l_0$ and $h_i \leq n - 1$, and show that it strictly decreases at every recursive call the procedure makes. Conclude that each procedure halts.

Solution.

- (b) A classmate claims that a procedure halts as soon as some integer computed from its arguments strictly decreases at every recursive call. Consider

```
def down(k):  
    if k == 0:  
        return 0  
    return down(k - 2)
```

- (i) State for which integers $k \geq 0$ the call `down(k)` halts, and give the sequence of arguments for one k on which it does not.
- (ii) The quantity k is an integer and strictly decreases at every call, so the classmate's claim would make `down` halt on every input. State which requirement in the definition of a measure fails here, and at which call it first fails.
- (iii) Change one line of `down` so that it halts on every integer k , and give a measure that proves your version halts.

Solution.

(c) A student changes the last line of `binary` to

```
return binary(A, mid, hi, t)
```

reasoning that `A[mid]` has already been ruled out, so leaving it in the portion is harmless. Show that the modified procedure fails to halt. Give a specific array A , a specific target t , and the sequence of calls it makes, and identify the exact point at which your measure from part (a) stops decreasing.

Solution.

- (d) Show that both procedures halt on the two boundary cases: an array of length 0, called as $lo = 0$, $hi = -1$; and an array of length 1. For each procedure and each case, give the calls made and the value returned. State which line of the procedure produces the return value.

Solution.

- (e) Halting and correctness are separate claims, and neither implies the other. Give two procedures for the searching problem, in pseudocode:
- (i) one that halts on every input but is not correct;
 - (ii) one that is correct whenever it returns, but does not halt on every input.

Neither needs to be useful, and neither may be one of the two procedures above. For each, say in one sentence which of the two claims fails and on what input.

Solution.

Problem 3. Correctness: does it return the right answer?**[25 points]**

Problem 2 established that both procedures halt. Assume that here, and prove only that what they return is correct.

Both proofs are inductions on the size of the portion still under consideration, $d := \text{hi} - \text{lo} + 1$. The base case of each proof is the base case of the procedure, where the portion is empty. The inductive hypothesis is applied to the call that the procedure makes.

- (a) State the claim to be proved, as a single precise sentence about the return value of `linear`($A, \text{lo}, \text{hi}, t$). It must cover both outcomes: what the returned index satisfies when t is present in $A[\text{lo} \dots \text{hi}]$, and what NOT-FOUND asserts when it is not. State it so that the base case, where the portion is empty, is an instance of the claim rather than an exception to it.

Solution.

- (b) Prove your claim from part (a) by induction on d . Label the base case, the inductive hypothesis, and the inductive step. In the step, say which line establishes the result when $A[\text{lo}] = t$ and where the inductive hypothesis is applied when it does not.

Solution.

Base case. *the smallest case, where the procedure returns without recursing*

Inductive hypothesis. *what you assume, and for which call*

Inductive step. *the argument, and where the hypothesis is used*

- (c) State and prove the corresponding claim for binary, by induction on d . The step now has three branches, and you must handle all three.

The step turns on a property that the base case alone cannot supply: if t occurs anywhere in A , then t occurs within $A[1o \dots hi]$. Show that each recursive call preserves it.

Solution.

Base case. *the smallest case, where the procedure returns without recursing*

Inductive hypothesis. *what you assume, and for which call*

Inductive step. *the argument, and where the hypothesis is used*

- (d) Your proof in part (c) used sortedness. Identify the single step of the argument that fails without it. Then give an array of length 5 that is not sorted, together with a target t present in that array, on which `binary` returns NOT-FOUND. Trace the calls.

Solution.

(e) Suppose t occurs more than once in A .

- (i) State what each procedure returns, and whether that violates the claim you stated in part (a). Answer for both procedures.
- (ii) Modify `binary` so that it returns the *smallest* index i with $A[i] = t$. Give the modified procedure in full.
- (iii) State the claim your modified procedure satisfies, and say which part of your proof in part (c) must change. A full proof is not required; identify the branch and the new argument it needs.

Solution.

A note on binary search

The idea of binary search appears in print by 1946. A version correct for arrays of arbitrary length was not published until 1962.

It has a documented history of incorrect implementation. Bentley, in *Programming Pearls*, reports that when the problem was set for groups of professional programmers, about ten percent produced a correct version. A fault in the midpoint computation went undetected in a widely distributed Java library for several years, because it is visible only on very large arrays. On a machine with fixed-width integers, $lo + hi$ can exceed the largest representable value even when both operands are valid indices. The expression $lo + (hi - lo) // 2$ cannot. Python's integers are unbounded, so the fault cannot arise in the code as written here. The proofs in Problem 3 do not detect it either, because they assume exact arithmetic. A proof is valid only for the machine model it assumes.

Problem 4. Time and space: what does the procedure cost?**[25 points]**

Let $T_{\text{lin}}(n)$ and $T_{\text{bin}}(n)$ be the worst-case number of comparisons on a portion of n elements. Count one comparison for each examination of an array element against t ; ignore the index test $\text{lo} > \text{hi}$.

Space is counted *beyond the array*, which is supplied by the caller and is not charged to the procedure. A recursive call that has not yet returned occupies memory. Its arguments and its position in the procedure are stored until the call it made returns. The number of such calls at one time is the *depth* of the recursion, and the extra space is proportional to the depth.

- (a) Write a recurrence for each procedure, with its base case. For binary, state the size of the larger of the two portions that remain after one comparison, and justify using $\lfloor \cdot \rfloor$ in the recurrence rather than $n/2$.

Solve both recurrences. Give $T_{\text{lin}}(n)$ and $T_{\text{bin}}(n)$ exactly as formulas in n , and each in Θ terms. Check your binary formula at $n = 1, 2, 3, 4, 7, 8, 15$ against the counts your program reported in Problem 1(b), and present the values in a table beside the formula. Then evaluate both formulas at $n = 10,000$.

Give the best case of each procedure as a formula in n , with an input achieving it. The two best cases are the same. State what that says about the usefulness of the best case for choosing between the procedures, and name the case that should be compared instead.

Solution.

- (b) Assume t occurs exactly once in A and is equally likely to be at any of the n positions. Let C be the number of comparisons `linear` makes. Show that $\mathbb{E}[C] = (n+1)/2$, using $\sum_{i=1}^n i = n(n+1)/2$ and the fact that t at position i costs i comparisons.

State $\mathbb{E}[C]$ in Θ terms and compare it to the worst case. Then answer, in two sentences: does averaging over inputs change which of the two procedures you would choose, and why?

Solution.

- (c) Give the maximum recursion depth of each procedure on an array of n elements, as a formula in n , and the extra space of each in Θ terms. Evaluate both depths at $n = 10^9$.

Account for the failure you observed in Problem 1(c) using these formulas, and state the largest n at which your recursive `linear` could run under an interpreter whose recursion depth is capped at 1,000. State why recursive `binary` did not fail at $n = 2^{20}$.

Give the extra space of the two loop versions you wrote in Problem 1(c), in Θ terms, and say whether the comparison count differs between the recursive and loop forms.

Then fill in a table with one row per procedure and one column for each of: worst-case comparisons, best-case comparisons, extra space as written recursively, extra space as a loop, and whether the procedure requires a sorted array.

Solution.

- (d) The records are still sorted, but are stored in a *linked list*: each record holds the location of the next one, the first record's location is known, and reaching the k -th record requires stepping through the $k - 1$ before it. Reading a record you have reached still costs one step.

State the running time of each procedure under this representation, in Θ terms, counting steps rather than comparisons. Identify the single operation whose cost changed and say what it cost before and after.

State which procedure has the larger change in running time, and give the reason in one sentence.

Solution.

- (e) A registry holds n records and will answer q searches.

Strategy A runs linear for each of the q searches on the unordered array. **Strategy B** sorts the array once, at a cost of $n \log_2 n$ comparisons, then runs binary for each search.

- (i) Write the total worst-case cost of each strategy as a function of n and q , taking every constant factor as 1. Solve for the break-even value q^* at which the two are equal, evaluate it at $n = 10^3$, 10^6 , and 10^9 , and state how q^* grows with n . State what the value implies about when sorting is worth its cost.
- (ii) The registry now admits new patients throughout the day. Give the cost of inserting one record into a sorted array of n records, separating the cost of *locating* the position from the cost of *making room* for it, and say which dominates.
- (iii) State, in two or three sentences, what property a storage scheme would need so that insertion, deletion, and search are all $O(\log n)$. Name the property; you are not asked to give the scheme, and no credit is attached to naming a particular data structure.

Solution.

Reference: CLRS, *Introduction to Algorithms*, 4th ed., Chapters 2–4. It contains solutions to parts of Problems 3 and 4; read it after you have attempted them.